SQL PROGRAMMING SOLUTIONS PORTFOLIO

Candidate Details

Candidate Name: Gaizal Sharma

College: Noida Institute of Engineering and Technology (NIET)

Branch: Bachelor of Technology (Computer Science & Engineering - Internet of Things)

Database: MySQL (ANSI SQL)

Purpose

This document contains SQL queries covering the most frequently asked database concepts for technical interviews and placement assessments. The portfolio demonstrates proficiency in SQL programming, database management, data retrieval, data manipulation, joins, aggregate functions, subqueries, constraints, and database design.

Skills Demonstrated

- **ANSI SQL**
- **Database Design**
- **Data Retrieval**
- **Data Manipulation**
- **Data Definition**
- **Aggregate Functions**
- **Joins**
- **Subqueries**
- **Constraints**
- **Views**
- **Query Optimization**
- **Problem Solving**

Topics Covered

Topic 1: DDL Commands

1. **Create Database**
2. **Create Table**

3. **Alter Table**
4. **Add Column**
5. **Rename Column**
6. **Modify Column**
7. **Drop Column**
8. **Truncate Table**
9. **Drop Table**

Topic 2: DML Commands

10. **Insert Record**
11. **Update Record**
12. **Delete Record**

Topic 3: DQL

13. **SELECT Statement**
14. **WHERE Clause**
15. **DISTINCT**
16. **ORDER BY**
17. **LIMIT**

Topic 4: Operators

18. **LIKE**
19. **IN**
20. **BETWEEN**
21. **IS NULL**
22. **AND / OR / NOT**

Topic 5: Aggregate Functions

23. **COUNT()**
24. **SUM()**
25. **AVG()**

26. **MIN()**
27. **MAX()**

Topic 6: GROUP BY & HAVING

28. **GROUP BY**
29. **HAVING**

Topic 7: Joins

30. **INNER JOIN**
31. **LEFT JOIN**
32. **RIGHT JOIN**
33. **FULL OUTER JOIN**
34. **SELF JOIN**

Topic 8: Subqueries

35. **Second Highest Salary**
36. **Employee with Maximum Salary**
37. **EXISTS**
38. **IN Subquery**

Topic 9: Constraints

39. **PRIMARY KEY**
40. **FOREIGN KEY**
41. **UNIQUE**
42. **NOT NULL**
43. **DEFAULT**

Topic 10: Advanced SQL

44. **CASE Statement**
45. **UNION**
46. **UNION ALL**

47. View

48. Index

49. Stored Procedure (Basic)

50. Trigger (Basic)

Database Used

Database: MySQL

Declaration

I hereby declare that all SQL queries included in this document were written and practiced by me for learning, interview preparation, and placement purposes. These solutions demonstrate my understanding of SQL programming and database concepts.

Candidate Name

Gaizal Sharma

---

1. Create Database

Problem Statement

Write an SQL query to create a new database named CompanyDB.

SQL Query

```
CREATE DATABASE CompanyDB;
```

Explanation

The CREATE DATABASE statement creates a new database with the specified name.

Sample Output

Database created successfully.

---

2. Create Table

Problem Statement

Write an SQL query to create an Employee table.

SQL Query

```
CREATE TABLE Employee
(
   emp_id INT PRIMARY KEY,
```

emp_name VARCHAR(50),

department VARCHAR(30),

salary DECIMAL(10,2)

);

Explanation

The CREATE TABLE statement creates a table named Employee with four columns.

Sample Output

Table created successfully.

---

3. Alter Table

Problem Statement

Write an SQL query to rename the Employee table to Employees.

SQL Query

ALTER TABLE Employee

RENAME TO Employees;

Explanation

The ALTER TABLE statement modifies an existing table. Here it renames the table.

Sample Output

Table renamed successfully.

---

4. Add Column

Problem Statement

Write an SQL query to add an Email column to the Employee table.

SQL Query

ALTER TABLE Employee

ADD Email VARCHAR(100);

Explanation

The ADD keyword is used with ALTER TABLE to add a new column.

Sample Output

Column added successfully.

---

5. Rename Column

Problem Statement

Write an SQL query to rename the column emp_name to employee_name.

SQL Query

ALTER TABLE Employee

RENAME COLUMN emp_name TO employee_name;

Explanation

The RENAME COLUMN statement changes the name of an existing column.

Sample Output

Column renamed successfully.

6. Modify Column

Problem Statement

Write an SQL query to modify the data type of the **salary** column to DECIMAL(12,2).

SQL Query

ALTER TABLE Employee

MODIFY salary DECIMAL(12,2);

Explanation

The MODIFY statement changes the data type or properties of an existing column.

Sample Output

Column modified successfully.

---

7. Drop Column

Problem Statement

Write an SQL query to remove the **Email** column from the Employee table.

SQL Query

ALTER TABLE Employee

DROP COLUMN Email;

Explanation

The DROP COLUMN statement permanently removes a column from a table.

Sample Output

Column dropped successfully.

---

8. Truncate Table

Problem Statement

Write an SQL query to delete all records from the Employee table while keeping the table structure.

SQL Query

TRUNCATE TABLE Employee;

Explanation

TRUNCATE removes all rows from the table but retains its structure, indexes, and constraints.

Sample Output

Table truncated successfully.

---

9. Drop Table

Problem Statement

Write an SQL query to delete the Employee table permanently.

SQL Query

DROP TABLE Employee;

Explanation

The DROP TABLE statement permanently removes the table along with all its data and structure.

Sample Output

Table dropped successfully.

---

10. Insert Record

Problem Statement

Write an SQL query to insert a new employee record into the Employee table.

SQL Query

INSERT INTO Employee

(emp_id, emp_name, department, salary)

VALUES

(101, 'Rahul', 'IT', 50000);

Explanation

The INSERT INTO statement is used to add a new row to the table.

Sample Output

1 row inserted successfully.

11. Update Record

Problem Statement

Write an SQL query to update the salary of an employee whose emp_id is **101**.

Sample Table

Employee

+--------+----------+------------+--------+

| emp_id | emp_name | department | salary |

+--------+----------+------------+--------+

| 101 | Rahul | IT | 50000 |

| 102 | Priya | HR | 45000 |

| 103 | Amit | IT | 60000 |

+--------+----------+------------+--------+

SQL Query

UPDATE Employee

SET salary = 55000

WHERE emp_id = 101;

Explanation

The UPDATE statement modifies existing records in a table. The WHERE clause ensures that only the specified employee's record is updated.

Sample Output

1 row updated successfully.

---

12. Delete Record

Problem Statement

Write an SQL query to delete the employee whose emp_id is **103**.

Sample Table

Employee

+--------+----------+------------+--------+

| emp_id | emp_name | department | salary |

+--------+----------+------------+--------+

| 101 | Rahul | IT | 55000 |

| 102 | Priya | HR | 45000 |

| 103 | Amit | IT | 60000 |

+--------+----------+------------+--------+

SQL Query

DELETE FROM Employee

WHERE emp_id = 103;

Explanation

The DELETE statement removes records from a table. The WHERE clause specifies which record should be deleted.

Sample Output

1 row deleted successfully.

---

13. SELECT Statement

Problem Statement

Write an SQL query to display all records from the Employee table.

Sample Table

Employee

+--------+----------+------------+--------+

| emp_id | emp_name | department | salary |

+--------+----------+------------+--------+

| 101 | Rahul | IT | 55000 |

| 102 | Priya | HR | 45000 |

+--------+----------+------------+--------+

SQL Query

```
SELECT * FROM Employee;
```

Explanation

The SELECT statement retrieves data from a table. The * symbol displays all columns.

Sample Output

```
+--------+----------+------------+--------+
| emp_id | emp_name | department | salary |
+--------+----------+------------+--------+
| 101    | Rahul    | IT         | 55000  |
| 102    | Priya    | HR         | 45000  |
+--------+----------+------------+--------+
```

---

14. WHERE Clause

Problem Statement

Write an SQL query to display employees who belong to the **IT** department.

Sample Table

Employee

```
+--------+----------+------------+--------+
| emp_id | emp_name | department | salary |
+--------+----------+------------+--------+
| 101    | Rahul    | IT         | 55000  |
| 102    | Priya    | HR         | 45000  |
| 103    | Amit     | IT         | 60000  |
+--------+----------+------------+--------+
```

SQL Query

```
SELECT *
FROM Employee
WHERE department = 'IT';
```

Explanation

The WHERE clause filters records based on a specified condition.

Sample Output

+--------+----------+------------+--------+

| emp_id | emp_name | department | salary |

+--------+----------+------------+--------+

| 101 | Rahul | IT | 55000 |

| 103 | Amit | IT | 60000 |

+--------+----------+------------+--------+

---

15. DISTINCT

Problem Statement

Write an SQL query to display all unique department names.

Sample Table

Employee

+--------+----------+------------+

| emp_id | emp_name | department |

+--------+----------+------------+

| 101 | Rahul | IT |

| 102 | Priya | HR |

| 103 | Amit | IT |

| 104 | Neha | Finance |

+--------+----------+------------+

SQL Query

SELECT DISTINCT department

FROM Employee;

Explanation

The DISTINCT keyword removes duplicate values and returns only unique records.

Sample Output

+------------+

| department |

+------------+

| IT |

| HR |

| Finance |

+------------+

21. AND / OR / NOT

Problem Statement

Write SQL queries using **AND**, **OR**, and **NOT** operators.

Sample Table

Employee

+--------+----------+------------+--------+

| emp_id | emp_name | department | salary |

+--------+----------+------------+--------+

| 101 | Rahul | IT | 55000 |

| 102 | Priya | HR | 45000 |

| 103 | Amit | IT | 70000 |

| 104 | Neha | Finance | 65000 |

+--------+----------+------------+--------+

SQL Query (AND)

```
SELECT *
FROM Employee
WHERE department = 'IT'
AND salary > 60000;
```

SQL Query (OR)

```
SELECT *
FROM Employee
WHERE department = 'HR'
OR department = 'Finance';
```

SQL Query (NOT)

```
SELECT *
```

FROM Employee

WHERE NOT department = 'IT';

Explanation

- **AND** → All conditions must be true.
- **OR** → At least one condition must be true.
- **NOT** → Reverses the condition.

Sample Output

AND

```
+--------+----------+------------+--------+
| 103   | Amit    | IT       | 70000  |
+--------+----------+------------+--------+
```

OR

```
+--------+----------+------------+--------+
| 102   | Priya   | HR       | 45000  |
| 104   | Neha    | Finance   | 65000  |
+--------+----------+------------+--------+
```

NOT

```
+--------+----------+------------+--------+
| 102   | Priya   | HR       | 45000  |
| 104   | Neha    | Finance   | 65000  |
+--------+----------+------------+--------+
```

---

22. COUNT()

Problem Statement

Write an SQL query to count the total number of employees.

Sample Table

Employee

```
+--------+----------+
```

| emp_id | emp_name |

+--------+----------+

| 101 | Rahul |

| 102 | Priya |

| 103 | Amit |

| 104 | Neha |

+--------+----------+

SQL Query

SELECT COUNT(*) AS Total_Employees

FROM Employee;

Explanation

The COUNT() function returns the total number of rows.

Sample Output

+-----------------+

| Total_Employees |

+-----------------+

| 4 |

+-----------------+

---

23. SUM()

Problem Statement

Write an SQL query to calculate the total salary of all employees.

Sample Table

Employee

+--------+----------+--------+

| emp_id | emp_name | salary |

+--------+----------+--------+

| 101 | Rahul | 55000 |

| 102 | Priya | 45000 |

| 103 | Amit | 70000 |

| 104 | Neha | 65000 |

+--------+----------+--------+

SQL Query

SELECT SUM(salary) AS Total_Salary

FROM Employee;

Explanation

The SUM() function calculates the total of a numeric column.

Sample Output

+--------------+

| Total_Salary |

+--------------+

| 235000 |

+--------------+

---

24. AVG()

Problem Statement

Write an SQL query to calculate the average salary of employees.

Sample Table

Employee

+--------+----------+--------+

| emp_id | emp_name | salary |

+--------+----------+--------+

| 101 | Rahul | 55000 |

| 102 | Priya | 45000 |

| 103 | Amit | 70000 |

| 104 | Neha | 65000 |

+--------+----------+--------+

SQL Query

SELECT AVG(salary) AS Average_Salary

FROM Employee;

Explanation

The AVG() function calculates the average value of a numeric column.

Sample Output

```
+----------------+
| Average_Salary |
+----------------+
|   58750.00   |
+----------------+
```

---

25. MIN()

Problem Statement

Write an SQL query to find the minimum salary in the Employee table.

Sample Table

Employee

```
+--------+----------+--------+
| emp_id | emp_name | salary |
+--------+----------+--------+
| 101    | Rahul    | 55000  |
| 102    | Priya    | 45000  |
| 103    | Amit     | 70000  |
| 104    | Neha     | 65000  |
+--------+----------+--------+
```

SQL Query

```
SELECT MIN(salary) AS Minimum_Salary
FROM Employee;
```

Explanation

The MIN() function returns the smallest value from a column.

Sample Output

```
+----------------+
| Minimum_Salary |
```

```
+----------------+
|    45000     |
+----------------+
```

26. MAX()

Problem Statement

Write an SQL query to find the maximum salary from the Employee table.

Sample Table

Employee

```
+--------+----------+--------+
| emp_id | emp_name | salary |
+--------+----------+--------+
| 101    | Rahul    | 55000  |
| 102    | Priya    | 45000  |
| 103    | Amit     | 70000  |
| 104    | Neha     | 65000  |
+--------+----------+--------+
```

SQL Query

```
SELECT MAX(salary) AS Maximum_Salary
FROM Employee;
```

Explanation

The MAX() function returns the largest value from a numeric column.

Sample Output

```
+----------------+
| Maximum_Salary |
+----------------+
|    70000     |
+----------------+
```

---

27. GROUP BY

Problem Statement

Write an SQL query to display the total salary department-wise.

Sample Table

Employee

```
+--------+----------+------------+--------+
| emp_id | emp_name | department | salary |
+--------+----------+------------+--------+
| 101    | Rahul    | IT         | 55000  |
| 102    | Priya    | HR         | 45000  |
| 103    | Amit     | IT         | 70000  |
| 104    | Neha     | HR         | 50000  |
| 105    | Arjun    | Finance    | 65000  |
+--------+----------+------------+--------+
```

SQL Query

```
SELECT department,
    SUM(salary) AS Total_Salary
FROM Employee
GROUP BY department;
```

Explanation

The GROUP BY clause groups rows having the same values and performs aggregate functions on each group.

Sample Output

```
+------------+--------------+
| department | Total_Salary |
+------------+--------------+
| Finance    | 65000        |
| HR         | 95000        |
| IT         | 125000       |
+------------+--------------+
```

---

28. HAVING

Problem Statement

Write an SQL query to display departments whose total salary is greater than **100000**.

Sample Table

Employee

```
+--------+----------+------------+--------+
| emp_id | emp_name | department | salary |
+--------+----------+------------+--------+
| 101    | Rahul    | IT         | 55000  |
| 102    | Priya    | HR         | 45000  |
| 103    | Amit     | IT         | 70000  |
| 104    | Neha     | HR         | 50000  |
| 105    | Arjun    | Finance    | 65000  |
+--------+----------+------------+--------+
```

SQL Query

```
SELECT department,
    SUM(salary) AS Total_Salary
FROM Employee
GROUP BY department
HAVING SUM(salary) > 100000;
```

Explanation

The HAVING clause filters grouped records after the GROUP BY operation.

Sample Output

```
+------------+--------------+
| department | Total_Salary |
+------------+--------------+
| IT         | 125000       |
+------------+--------------+
```

---

29. INNER JOIN

Problem Statement

Write an SQL query to display employee names along with their department names using **INNER JOIN**.

Sample Tables

Employee

+--------+----------+---------+

| emp_id | emp_name | dept_id |

+--------+----------+---------+

| 101 | Rahul | 1 |

| 102 | Priya | 2 |

| 103 | Amit | 1 |

+--------+----------+---------+

Department

+---------+-----------+

| dept_id | dept_name |

+---------+-----------+

| 1 | IT |

| 2 | HR |

+---------+-----------+

SQL Query

```
SELECT e.emp_name,
    d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

Explanation

INNER JOIN returns only the matching records from both tables.

Sample Output

+----------+-----------+

| emp_name | dept_name |

+----------+-----------+

| Rahul | IT |

| Priya | HR |

| Amit | IT |

+----------+-----------+

---

30. LEFT JOIN

Problem Statement

Write an SQL query to display all employees along with their department names. Employees without a matching department should also be displayed.

Sample Tables

Employee

+--------+----------+---------+

| emp_id | emp_name | dept_id |

+--------+----------+---------+

| 101 | Rahul | 1 |

| 102 | Priya | 2 |

| 103 | Amit | 3 |

+--------+----------+---------+

Department

+---------+-----------+

| dept_id | dept_name |

+---------+-----------+

| 1 | IT |

| 2 | HR |

+---------+-----------+

SQL Query

```
SELECT e.emp_name,
    d.dept_name
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.dept_id;
```

Explanation

LEFT JOIN returns all records from the left table (Employee) and the matching records from the right table (Department). If no match exists, NULL is returned for the right table columns.

Sample Output

+----------+-----------+

| emp_name | dept_name |

+----------+-----------+

| Rahul | IT |

| Priya | HR |

| Amit | NULL |

+----------+-----------+

31. RIGHT JOIN

Problem Statement

Write an SQL query to display all departments along with their employees. Departments without employees should also be displayed.

Sample Tables

Employee

+--------+----------+---------+

| emp_id | emp_name | dept_id |

+--------+----------+---------+

| 101 | Rahul | 1 |

| 102 | Priya | 2 |

+--------+----------+---------+

Department

+---------+-----------+

| dept_id | dept_name |

+---------+-----------+

| 1 | IT |

| 2 | HR |

| 3 | Finance |

+---------+-----------+

SQL Query

```
SELECT e.emp_name,
    d.dept_name
FROM Employee e
RIGHT JOIN Department d
ON e.dept_id = d.dept_id;
```

Explanation

RIGHT JOIN returns all records from the right table (Department) and the matching records from the left table (Employee). If no employee exists for a department, NULL is returned.

Sample Output

+----------+-----------+

| emp_name | dept_name |

+----------+-----------+

| Rahul | IT |

| Priya | HR |

| NULL | Finance |

+----------+-----------+

---

32. FULL OUTER JOIN

Problem Statement

Write an SQL query to display all employees and all departments, including unmatched records from both tables.

Sample Tables

Employee

+--------+----------+---------+

| emp_id | emp_name | dept_id |

+--------+----------+---------+

| 101 | Rahul | 1 |

| 102 | Priya | 2 |

| 103 | Amit | 5 |

+--------+----------+---------+

Department

+---------+-----------+

| dept_id | dept_name |

+---------+-----------+

| 1 | IT |

| 2 | HR |

| 3 | Finance |

+---------+-----------+

SQL Query (Standard SQL)

```
SELECT e.emp_name,
    d.dept_name
FROM Employee e
FULL OUTER JOIN Department d
ON e.dept_id = d.dept_id;
```

MySQL Note

MySQL does **not** support FULL OUTER JOIN directly. It can be simulated using LEFT JOIN, RIGHT JOIN, and UNION.

```
SELECT e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d
ON e.dept_id = d.dept_id

UNION

SELECT e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d
ON e.dept_id = d.dept_id;
```

Explanation

A FULL OUTER JOIN returns all matching and non-matching records from both tables.

Sample Output

```
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Rahul    | IT        |
| Priya    | HR        |
| Amit     | NULL      |
| NULL     | Finance   |
+----------+-----------+
```

---

33. SELF JOIN

Problem Statement

Write an SQL query to display employees along with their managers using a Self Join.

Sample Table

Employee

```
+--------+----------+------------+
| emp_id | emp_name | manager_id |
```

+--------+----------+------------+

| 1 | Rahul | NULL |

| 2 | Priya | 1 |

| 3 | Amit | 1 |

| 4 | Neha | 2 |

+--------+----------+------------+

SQL Query

```
SELECT e.emp_name AS Employee,
    m.emp_name AS Manager
FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;
```

Explanation

A SELF JOIN joins a table with itself. Here, one copy represents employees and the other represents managers.

Sample Output

+----------+---------+

| Employee | Manager |

+----------+---------+

| Rahul | NULL |

| Priya | Rahul |

| Amit | Rahul |

| Neha | Priya |

+----------+---------+

---

34. Second Highest Salary

Problem Statement

Write an SQL query to find the second highest salary from the Employee table.

Sample Table

Employee

```
+--------+----------+--------+
| emp_id | emp_name | salary |
+--------+----------+--------+
| 101    | Rahul    | 55000  |
| 102    | Priya    | 70000  |
| 103    | Amit     | 65000  |
| 104    | Neha     | 80000  |
+--------+----------+--------+
```

SQL Query

```
SELECT MAX(salary) AS Second_Highest_Salary
FROM Employee
WHERE salary <
(
  SELECT MAX(salary)
  FROM Employee
);
```

Explanation

The inner query finds the highest salary. The outer query returns the highest salary that is less than the maximum salary.

Sample Output

```
+------------------------+
| Second_Highest_Salary  |
+------------------------+
| 70000                  |
+------------------------+
```

---

35. Employee with Maximum Salary

Problem Statement

Write an SQL query to display the employee(s) having the highest salary.

Sample Table

Employee

+--------+----------+--------+

| emp_id | emp_name | salary |

+--------+----------+--------+

| 101 | Rahul | 55000 |

| 102 | Priya | 80000 |

| 103 | Amit | 65000 |

| 104 | Neha | 80000 |

+--------+----------+--------+

SQL Query

```
SELECT *
FROM Employee
WHERE salary =
(
  SELECT MAX(salary)
  FROM Employee
);
```

Explanation

The subquery finds the highest salary, and the outer query returns all employees whose salary matches that value.

Sample Output

+--------+----------+--------+

| emp_id | emp_name | salary |

+--------+----------+--------+

| 102 | Priya | 80000 |

| 104 | Neha | 80000 |

+--------+----------+--------+

36. EXISTS

Problem Statement

Write an SQL query to display employees whose department exists in the Department table using the EXISTS operator.

Sample Tables

Employee

```
+--------+----------+---------+
| emp_id | emp_name | dept_id |
+--------+----------+---------+
| 101    | Rahul    | 1       |
| 102    | Priya    | 2       |
| 103    | Amit     | 4       |
+--------+----------+---------+
```

Department

```
+---------+-----------+
| dept_id | dept_name |
+---------+-----------+
| 1       | IT        |
| 2       | HR        |
| 3       | Finance   |
+---------+-----------+
```

SQL Query

```
SELECT *
FROM Employee e
WHERE EXISTS
(
   SELECT 1
   FROM Department d
   WHERE d.dept_id = e.dept_id
);
```

Explanation

The EXISTS operator returns **TRUE** if the subquery returns at least one row. Here, only employees whose department exists in the Department table are displayed.

Sample Output

+--------+----------+---------+

| emp_id | emp_name | dept_id |

+--------+----------+---------+

| 101 | Rahul | 1 |

| 102 | Priya | 2 |

+--------+----------+---------+

---

37. IN Subquery

Problem Statement

Write an SQL query to display employees working in departments located in Delhi.

Sample Tables

Employee

+--------+----------+---------+

| emp_id | emp_name | dept_id |

+--------+----------+---------+

| 101 | Rahul | 1 |

| 102 | Priya | 2 |

| 103 | Amit | 3 |

+--------+----------+---------+

Department

+---------+-----------+----------+

| dept_id | dept_name | location |

+---------+-----------+----------+

| 1 | IT | Delhi |

| 2 | HR | Mumbai |

| 3 | Finance | Delhi |

+---------+-----------+----------+

SQL Query

```
SELECT *
FROM Employee
WHERE dept_id IN
(
  SELECT dept_id
  FROM Department
  WHERE location = 'Delhi'
);
```

Explanation

The subquery returns all department IDs located in Delhi. The outer query retrieves employees whose department ID matches one of those values.

Sample Output

+--------+----------+---------+

| emp_id | emp_name | dept_id |

+--------+----------+---------+

| 101 | Rahul | 1 |

| 103 | Amit | 3 |

+--------+----------+---------+

---

38. PRIMARY KEY

Problem Statement

Write an SQL query to create a table with a **PRIMARY KEY**.

SQL Query

```
CREATE TABLE Employee
(
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
```

```
  department VARCHAR(30),
  salary DECIMAL(10,2)
);
```

Explanation

A **PRIMARY KEY** uniquely identifies each record in a table.

- Cannot contain NULL
- Must contain unique values
- Only one primary key is allowed per table

Sample Output

Table created successfully.

---

39. FOREIGN KEY

Problem Statement

Write an SQL query to create two tables using a **FOREIGN KEY** relationship.

SQL Query

```
CREATE TABLE Department
(
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(50)
);

CREATE TABLE Employee
(
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  dept_id INT,
  FOREIGN KEY (dept_id)
  REFERENCES Department(dept_id)
);
```

Explanation

A **FOREIGN KEY** establishes a relationship between two tables and maintains referential integrity.

Sample Output

Tables created successfully.

---

40. UNIQUE Constraint

Problem Statement

Write an SQL query to create a table where the email address must be unique.

SQL Query

```
CREATE TABLE Employee
(
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  email VARCHAR(100) UNIQUE
);
```

Explanation

The **UNIQUE** constraint ensures that duplicate values are not allowed in the specified column, while allowing NULL values (behavior may vary slightly by DBMS).

Sample Output

Table created successfully.

41. NOT NULL Constraint

Problem Statement

Write an SQL query to create a table where the employee name cannot be NULL.

SQL Query

```
CREATE TABLE Employee
(
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50) NOT NULL,
  department VARCHAR(30),
  salary DECIMAL(10,2)
);
```

Explanation

The **NOT NULL** constraint ensures that a column must always contain a value. A NULL value cannot be inserted into that column.

Sample Output

Table created successfully.

---

42. DEFAULT Constraint

Problem Statement

Write an SQL query to assign a default value of **'Active'** to the employee status.

SQL Query

```
CREATE TABLE Employee
(
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  status VARCHAR(20) DEFAULT 'Active'
);
```

Explanation

The **DEFAULT** constraint automatically assigns a predefined value when no value is provided during insertion.

Sample Output

Table created successfully.

---

43. CASE Statement

Problem Statement

Write an SQL query to classify employees based on their salary.

Sample Table

Employee

```
+--------+----------+--------+
| emp_id | emp_name | salary |
+--------+----------+--------+
| 101    | Rahul    | 45000  |
```

| 102 | Priya | 70000 |

| 103 | Amit | 90000 |

+--------+----------+--------+

SQL Query

```
SELECT emp_name,
    salary,
    CASE
      WHEN salary >= 80000 THEN 'High'
      WHEN salary >= 60000 THEN 'Medium'
      ELSE 'Low'
    END AS Salary_Level
FROM Employee;
```

Explanation

The **CASE** statement works like an IF-ELSE statement and returns values based on specified conditions.

Sample Output

+----------+--------+--------------+

| emp_name | salary | Salary_Level |

+----------+--------+--------------+

| Rahul | 45000 | Low |

| Priya | 70000 | Medium |

| Amit | 90000 | High |

+----------+--------+--------------+

---

44. UNION

Problem Statement

Write an SQL query to combine employee names from two tables without duplicate records.

Sample Tables

Employee2025

+----------+

| emp_name |

+----------+

| Rahul |

| Priya |

| Amit |

+----------+

Employee2026

+----------+

| emp_name |

+----------+

| Amit |

| Neha |

| Arjun |

+----------+

SQL Query

```
SELECT emp_name
FROM Employee2025

UNION

SELECT emp_name
FROM Employee2026;
```

Explanation

The **UNION** operator combines the results of two or more SELECT statements and automatically removes duplicate records.

Sample Output

+----------+

| emp_name |

+----------+

| Rahul |

| Priya |

| Amit |

| Neha |

| Arjun |

+----------+

---

45. UNION ALL

Problem Statement

Write an SQL query to combine employee names from two tables including duplicate records.

Sample Tables

Employee2025

+----------+

| emp_name |

+----------+

| Rahul |

| Priya |

| Amit |

+----------+

Employee2026

+----------+

| emp_name |

+----------+

| Amit |

| Neha |

| Arjun |

+----------+

SQL Query

```
SELECT emp_name
FROM Employee2025

UNION ALL

SELECT emp_name
FROM Employee2026;
```

Explanation

The **UNION ALL** operator combines the results of multiple SELECT statements **without removing duplicate records**, making it faster than UNION because no duplicate check is performed.

Sample Output

```
+----------+
| emp_name |
+----------+
| Rahul    |
| Priya    |
| Amit     |
| Amit     |
| Neha     |
| Arjun    |
+----------+
```

46. View

Problem Statement

Write an SQL query to create a view that displays employee names and salaries.

Sample Table

Employee

```
+--------+----------+--------+
| emp_id | emp_name | salary |
+--------+----------+--------+
```

| 101 | Rahul | 55000 |

| 102 | Priya | 70000 |

| 103 | Amit | 65000 |

+--------+----------+--------+

SQL Query

CREATE VIEW Employee_View AS

SELECT emp_name, salary

FROM Employee;

Explanation

A **VIEW** is a virtual table created from the result of an SQL query. It does not store data itself but displays data from one or more underlying tables.

Sample Output

SELECT * FROM Employee_View;

+----------+--------+

| emp_name | salary |

+----------+--------+

| Rahul | 55000 |

| Priya | 70000 |

| Amit | 65000 |

+----------+--------+

---

47. Index

Problem Statement

Write an SQL query to create an index on the **emp_name** column.

SQL Query

CREATE INDEX idx_emp_name

ON Employee(emp_name);

Explanation

An **INDEX** improves the speed of data retrieval operations by creating a lookup structure on one or more columns. However, indexes require additional storage and can slightly slow down INSERT, UPDATE, and DELETE operations.

Sample Output

Index created successfully.

---

48. Stored Procedure (Basic)

Problem Statement

Write an SQL stored procedure to display all employee records.

SQL Query

```
DELIMITER //

CREATE PROCEDURE GetEmployees()

BEGIN

  SELECT *
  FROM Employee;

END //

DELIMITER ;
```

Explanation

A **Stored Procedure** is a precompiled collection of SQL statements stored in the database. It can be executed whenever required, improving code reusability and reducing repetitive SQL statements.

Execute Procedure

```
CALL GetEmployees();
```

Sample Output

```
+--------+----------+------------+--------+
| emp_id | emp_name | department | salary |
+--------+----------+------------+--------+
| 101    | Rahul    | IT         | 55000  |
| 102    | Priya    | HR         | 70000  |
| 103    | Amit     | Finance    | 65000  |
+--------+----------+------------+--------+
```

---

49. Trigger (Basic)

Problem Statement

Write an SQL trigger to automatically store employee details in an audit table whenever a new employee is inserted.

SQL Query

```
CREATE TABLE Employee_Audit
(
  emp_id INT,
  action_time TIMESTAMP
);

DELIMITER //

CREATE TRIGGER Employee_Insert

AFTER INSERT ON Employee

FOR EACH ROW

BEGIN

  INSERT INTO Employee_Audit
  VALUES
  (
    NEW.emp_id,
    NOW()
  );

END //
```

DELIMITER ;

Explanation

A **Trigger** is a database object that automatically executes when a specified event (INSERT, UPDATE, or DELETE) occurs on a table.

Sample Output

Trigger created successfully.

---

50. Difference between DELETE, TRUNCATE and DROP

Problem Statement

Differentiate between **DELETE**, **TRUNCATE**, and **DROP** commands.

Feature	DELETE	TRUNCATE	DROP
Command Type	DML	DDL	DDL
Removes	Selected Rows	All Rows	Entire Table
WHERE Clause	✅ Yes	❌ No	❌ No
Table Structure	Remains	Remains	Deleted
Rollback	✅ Possible (before commit)	❌ Generally not	❌ No
Speed	Slow	Fast	Fastest

Examples

DELETE

```
DELETE FROM Employee
WHERE emp_id = 101;
```

TRUNCATE

```
TRUNCATE TABLE Employee;
```

DROP

```
DROP TABLE Employee;
```

Explanation

- **DELETE removes specific or all rows while keeping the table structure intact.**
- **TRUNCATE removes all rows but retains the table structure. It is faster than DELETE.**
- **DROP removes the entire table, including its structure, indexes, constraints, and data.**